QUESTION 1

- What is the difference between verification and validation?
  - Verification is demonstrating conformance to a specification whereas validation is checking that a system meets the customer's needs.
- Why validation is particularly difficult process?
  - Validation is difficult because there are many different stakeholders who may use the system with different needs. Therefore, a system that meets one user's needs may not meet the needs of a different user. Furthermore, needs change as a system is developed so the needs as identified when the system was specified may be different by the time that the system is tested.

QUESTION 2

- Why it is NOT necessary for a program to be completely free of defects before it is delivered to its customers?

  1. Remaining defects are minor defects that do not cause system corruption and which are transient i.e. which can be cleared when new data is input.
  2. Remaining defects are such that they are recoverable and a recovery function that causes minimum user disruption is available.
  3. The benefits to the customer's business from the system exceed the problems that might be caused by remaining system defects.

- To what extent can testing be used to validate that the program for its purpose?
  - Testing cannot completely validate that a system is fit for its intended purpose as this requires a detailed knowledge of what that purpose will be and exactly how the system will be used.
  - As these details unavoidably change between deciding to procure a system and deploying that system, the testing will be necessarily incomplete. In addition, it is practically impossible for all except trivial system to have a complete test set that covers all possible ways that the system is likely to be used.

QUESTION 3

- Why program inspections are an effective technique for discovering errors in a program?

  1. They can find several faults in one pass without being concerned about interference between program faults.
  2. They bring a number of people with different experience of different types of errors. Hence, the team approach offers greater coverage than any individual can bring.
  3. They force the program author to re-examine the program in detail - this often reveals errors or misunderstandings.

- What types of error are unlikely to be discovered through inspections?
  - The types of errors that inspections are unlikely to find are specification errors or errors that are based on a misunderstanding of the application domain (unless there are domain experts in the team).

QUESTION 4

- Why an organization with a competitive, elitist culture would probably find it difficult to introduce program inspections as a V&V technique?

  1. Program authors are unlikely to be open about their program because it exposes them to competition. The best programmers, in particular, are unlikely to want to be involved in inspections.
  2. The inspection team, rather than being cooperative, are likely to try to compete with each other to find the most errors. While competition can sometimes be helpful, in this situation if someone is obviously falling behind they may then stop participating actively in the inspection.

QUESTION 5

- What kind of programming errors could not be detected by a compiler but that might be detected in a program inspection?
  - **Logic errors :** A logic error is an error in the way a program works. The program can run but does not do what it is expected to do.
  - Logic errors can be caused by the programmer:
    - incorrectly using logical operators, eg expecting a program to stop when the value of a variable reaches 5, but using <5 instead of <=5
    - incorrectly using **Boolean operators**
    - unintentionally creating a situation where an **infinite loop** may occur
    - incorrectly using brackets in calculations
    - unintentionally using the same variable name at different points in the program for different purposes
    - using incorrect program design
- What kind of programming errors could not be detected by a compiler but that might be detected in a program inspection?
  - **Runtime errors:** A **runtime error** is an error that takes place during the running of a program.
  - An example is writing a program that tries to access the sixth item in an **array** that only contains five items. A runtime error is likely to crash the program.

QUESTION 6

- What type of faults are checked in an inspection?
  - Data Faults
  - Control Faults
  - Input/Output Faults
  - Interface Faults
  - Storage Management Faults
  - Exception Management Faults

QUESTION 7

- Why it may be cost effective to use formal methods in development of safety critical software systems?
  - Formal methods can be cost-effective in the development of safety-critical software systems because the costs of system failure are very high and so additional cost in the development process is justified. Most safety-critical systems have to gain regulatory approval before they are used and it is a very expensive process to convince a regulator that a system is safe. The use of a formal specification and associated correctness argument may be less than the costs e.g. of additional testing to convince the regulator of the safety of the system.
- Why do you think that some developers of this type of system are against the use of formal methods?
  - Because they are unfamiliar with the technology and unconvinced that a formal specification can be complete representation of the system. Furthermore, the problem with formal specifications are that they cannot be understood by system customers so they may conceo errors and give a false picture of the correctness of the system.

QUESTION 8

- What are the key strategies of Cleanroom Approach?
  - Formal Specification
  - Incremental Development
  - Structured Programming
  - Static Verification
  - Statistical Testing of the System

QUESTION 9

- What are the distinct types of testing?
  - Validation Testing
  - Defect Testing